

Vamos con el **Sprint 5**.
Este sprint es el núcleo diferenciador del MVP porque activa la lógica tipo “Waze”: **los ciudadanos cercanos validan reportes de otros usuarios**.

Sprint 5 — Validación ciudadana cercana y confirmación automática

Duración

1 semana: lunes a viernes

Objetivo del sprint

Implementar el sistema de validación cruzada ciudadana para que un usuario pueda confirmar incidentes cercanos reportados por otros vecinos, evitando autovalidación, evitando validaciones duplicadas y cambiando automáticamente el estado del incidente de “**Pendiente**” a “**Confirmado**” cuando alcance cinco validaciones válidas.

Sprint Goal

Al finalizar el Sprint 5, QHALI debe permitir que ciudadanos cercanos confirmen incidentes pendientes de otros usuarios, registrar esas validaciones de forma segura y actualizar automáticamente el estado del incidente cuando alcance cinco confirmaciones válidas.

Historias de usuario trabajadas en Sprint 5

Código	Historia de usuario	Prioridad
HU-19	Como ciudadano, quiero ver incidentes pendientes cercanos para confirmar si realmente existen.	P0
HU-20	Como ciudadano, quiero confirmar un incidente de otro usuario para ayudar a validar su veracidad.	P0
HU-21	Como sistema, quiero impedir que el creador valide su propio incidente para evitar manipulación.	P0
HU-22	Como sistema, quiero impedir que un usuario valide dos veces el mismo incidente para evitar duplicidad.	P0

Código	Historia de usuario	Prioridad
HU-23	Como ciudadano, quiero ver cuántas validaciones tiene un incidente para conocer su nivel de respaldo.	P0
HU-24	Como sistema, quiero cambiar automáticamente un incidente a “Confirmado” al llegar a cinco validaciones distintas.	P0
HU-25	Como ciudadano, quiero ver que el mapa cambie el color del incidente cuando pasa a confirmado.	P1

Entregables obligatorios del Sprint 5

Entregable	Responsable principal	Estado esperado
Tabla/modelo validations	Backend/API	Funcional
Endpoint GET /incidents/nearby	Backend/API + GeoData	Funcional
Endpoint POST /incidents/{id}/validate	Backend/API	Funcional
Bloqueo de autovalidación	Backend/API	Funcional
Bloqueo de validación duplicada	Backend/API	Funcional
Conteo de validaciones por incidente	Backend/API	Funcional
Cambio automático a “Confirmado” con 5 validaciones	Backend/API	Funcional
Vista de incidentes cercanos	Frontend PWA	Funcional
Botón “Confirmar incidente”	Frontend PWA	Funcional
Actualización visual del conteo	Frontend PWA	Funcional
Pruebas con varios usuarios demo	QA Técnico	Ejecutadas
Validación de distancia de 300 metros	GeoData/IA	Funcional

Product Backlog seleccionado para Sprint 5

Código	Ítem	Prioridad	Responsable
PB-47	Crear modelo Validation	P0	Backend/API

Código	Ítem	Prioridad	Responsable
PB-48	Crear endpoint de incidentes cercanos	P0	Backend/API + GeoData
PB-49	Crear función de cálculo de distancia	P0	GeoData/IA
PB-50	Crear endpoint para validar incidente	P0	Backend/API
PB-51	Bloquear autovalidación	P0	Backend/API
PB-52	Bloquear validación duplicada	P0	Backend/API
PB-53	Contar validaciones por incidente	P0	Backend/API
PB-54	Cambiar estado a “Confirmado” con 5 validaciones	P0	Backend/API
PB-55	Crear vista de alertas cercanas	P0	Frontend PWA
PB-56	Crear botón “Confirmar incidente”	P0	Frontend PWA
PB-57	Actualizar mapa y detalle con conteo de validaciones	P1	Frontend PWA
PB-58	Probar flujo con mínimo 5 usuarios distintos	P0	Scrum Master / QA

Reglas técnicas del Sprint 5

Regla 1 — Incidentes cercanos

El sistema debe mostrar incidentes pendientes dentro de un radio inicial de:

```
300 metros
```

Regla 2 — Solo reportes de otros usuarios

Un usuario no debe poder validar un incidente creado por él mismo.

```
incident.user_id != current_user.id
```

Regla 3 — Validación única por usuario

Un usuario solo puede validar una vez el mismo incidente.

```
unique(incident_id, user_id)
```

Regla 4 — Confirmación automática

Cuando un incidente llegue a cinco validaciones válidas:

```
status = "Confirmado"
```

Regla 5 — Reportes válidos para validación

Solo se pueden validar reportes en estado:

Pendiente

No se validan reportes:

Confirmado
En revisión
Resuelto

Lunes — Diseño de reglas de validación y modelo técnico

Objetivo del día

Cerrar las reglas exactas de validación ciudadana, crear la estructura técnica de validaciones y preparar los casos de prueba con varios usuarios.

Persona 1 — Product Owner Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar HU-19 a HU-25 y confirmar alcance obligatorio del sprint.	P0	Historias confirmadas.	Define qué se construye esta semana.
2	Confirmar radio inicial de validación cercana: 300 metros.	P0	Radio aprobado.	GeoData lo implementa.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
3	Confirmar regla de 5 validaciones para cambiar a “Confirmado”.	P0	Umbral aprobado.	Backend lo implementa.
4	Definir texto del botón: “Confirmar incidente”.	P0	Texto oficial.	Frontend lo usa.
5	Definir mensaje cuando el usuario ya validó: “Ya confirmaste este incidente”.	P1	Mensaje aprobado.	Frontend lo muestra.
6	Definir mensaje para autovalidación: “No puedes confirmar tu propio reporte”.	P1	Mensaje aprobado.	Frontend/Backend lo usan.
7	Confirmar que no se agregará reputación de usuarios en este sprint.	P0	Alcance protegido.	Evita sobrecarga.
8	Confirmar que no se agregarán comentarios ciudadanos en este sprint.	P0	Alcance protegido.	Mantiene foco en validación.
9	Actualizar criterios de aceptación de validación cruzada.	P0	Criterios listos para QA.	QA los prueba desde martes.

Persona 2 — Scrum Master / QA Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Crear caso de prueba: usuario B ve reporte pendiente creado por usuario A si está cerca.	P0	Prueba de cercanía definida.	Depende del radio de 300 m.
2	Crear caso de prueba: usuario B no ve reporte si está fuera de 300 m.	P0	Prueba de distancia definida.	Depende de GeoData.
3	Crear caso de prueba: usuario A no puede validar su propio reporte.	P0	Prueba de autovalidación definida.	Depende del backend.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
4	Crear caso de prueba: usuario B no puede validar dos veces el mismo reporte.	P0	Prueba de duplicidad definida.	Depende del modelo Validation.
5	Crear caso de prueba: 5 usuarios distintos validan y estado cambia a Confirmado.	P0	Prueba central definida.	Cierra el Sprint Goal.
6	Crear caso de prueba: reporte Confirmado ya no aparece como pendiente validable.	P0	Prueba posterior al cambio de estado.	Depende de endpoint nearby.
7	Crear usuarios demo mínimos: A, B, C, D, E, F.	P0	Usuarios listos para pruebas.	Necesarios para validar el umbral 5.
8	Crear matriz de pruebas por usuario, incidente y resultado esperado.	P1	QA ordenado.	Evita pruebas improvisadas.
9	Actualizar tablero Scrum con tareas del Sprint 5.	P0	Sprint Backlog asignado.	Control semanal.

Persona 3 — Developer Frontend PWA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar pantalla /map y modal de detalle creados en Sprint 4.	P0	Base de visualización identificada.	Se reutiliza para validar.
2	Definir ubicación de la sección “Incidentes cercanos”.	P0	Puede estar en /map o sección aparte.	Depende de criterio PO.
3	Crear componente NearbyIncidentsList .	P0	Lista visual preparada.	Consumirá endpoint nearby.
4	Crear componente ValidateButton .	P0	Botón reutilizable preparado.	Se conectará al endpoint.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
5	Crear estado local para validaciones: loading, success, error.	P0	UI preparada para interacción.	Evita clicks dobles.
6	Preparar visualización de <code>validation_count</code> en detalle de incidente.	P0	Conteo visible en UI.	Depende de backend.
7	Preparar estado visual para incidente ya validado.	P1	Botón deshabilitado o mensaje.	Depende de respuesta API.
8	Preparar estado visual para incidente confirmado.	P1	Cambio de color o etiqueta.	Conecta con mapa.
9	Revisar que usuario no autenticado no acceda a validaciones.	P0	Ruta protegida.	Depende del Sprint 2.

Persona 4 — Developer Backend/API

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar modelo <code>Incident</code> y campos necesarios para validación.	P0	Modelo listo para relación.	Base del Sprint 5.
2	Definir modelo <code>Validation</code> : <code>id</code> , <code>incident_id</code> , <code>user_id</code> , <code>created_at</code> .	P0	Modelo aprobado.	Se implementa martes.
3	Definir restricción única <code>incident_id + user_id</code> .	P0	Regla antduplicado clara.	Evita doble validación.
4	Definir contrato <code>GET /incidents/nearby?lat=&lng=&radius=</code> .	P0	Endpoint diseñado.	Frontend/GeoData dependen de esto.
5	Definir contrato <code>POST /incidents/{id}/validate</code> .	P0	Endpoint diseñado.	Frontend consume este endpoint.
6	Definir errores del endpoint de validación: propio reporte, duplicado, no encontrado, estado inválido.	P0	Errores controlados.	QA los probará.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
7	Definir cómo se contará <code>validation_count</code> .	P0	Conteo por incidente definido.	Frontend lo muestra.
8	Definir actualización automática de estado al llegar a 5.	P0	Regla interna lista.	Tarea central.
9	Documentar contratos iniciales en Swagger/README.	P1	Frontend puede integrarse sin ambigüedad.	Facilita miércoles.

Persona 5 — Developer GeoData/IA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar función de distancia preparada en Sprint 1/Sprint 3.	P0	Función identificada.	Base para nearby.
2	Validar fórmula Haversine o estrategia equivalente.	P0	Cálculo de distancia definido.	Backend lo usará.
3	Confirmar radio operativo de 300 metros.	P0	Parámetro base definido.	Endpoint nearby depende de esto.
4	Preparar coordenadas simuladas de usuarios validadores dentro de 300 m.	P0	Usuarios B-F con ubicaciones válidas.	QA las usa.
5	Preparar coordenadas simuladas fuera de 300 m.	P0	Caso negativo listo.	QA lo usa.
6	Definir estructura de respuesta geográfica: distancia en metros opcional.	P1	Campo <code>distance_meters</code> definido si se incluye.	Mejora frontend.
7	Verificar que incidentes existentes tengan coordenadas válidas.	P0	No se rompe endpoint nearby.	Depende Sprint 4.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
8	Preparar dataset de incidentes pendientes cercanos.	P0	Datos listos para pruebas.	Backend y QA lo usan.
9	Documentar regla geo: pendiente + radio + no propio.	P0	Regla formalizada.	Todos la siguen.

Resultado obligatorio del lunes

Al cerrar lunes debe existir:

- reglas de validación cerradas
- historias del Sprint 5 confirmadas
- modelo `Validation` diseñado
- contratos de endpoints definidos
- usuarios demo planificados
- coordenadas de prueba preparadas
- componentes frontend planificados
- checklist QA completo

Martes — Implementación del modelo `Validation` y endpoint de incidentes cercanos

Objetivo del día

Crear la base técnica de validación: tabla/modelo `validations`, endpoint para incidentes cercanos y función de distancia operativa.

Persona 1 — Product Owner Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar que la lista de incidentes cercanos solo muestre reportes pendientes.	P0	Criterio de visibilidad validado.	Depende del endpoint nearby.
2	Revisar que los incidentes propios no aparezcan como validables.	P0	Criterio funcional validado.	Backend/GeoData lo implementan.
3	Validar que el listado cercano no sea visualmente igual al historial privado.	P1	Diferencia de propósito clara.	Guía al Frontend.
4	Validar que <code>validation_count</code> se muestre aunque sea cero.	P0	Conteo visible desde inicio.	Backend/Frontend lo aplican.
5	Validar textos de estado: “Pendiente” y “Confirmado”.	P0	Consistencia visual.	Frontend lo implementa.
6	Revisar si el endpoint nearby necesita ubicación actual del usuario en frontend.	P0	Flujo confirmado.	Frontend debe capturar GPS.
7	Confirmar que el radio será configurable internamente, no por el usuario aún.	P1	Alcance controlado.	Evita UI extra.
8	Revisar avance contra Sprint Goal.	P0	Riesgos detectados.	Control temprano.
9	Ajustar criterios de aceptación si aparece restricción técnica.	P1	QA actualizado.	Evita inconsistencias.

Persona 2 — Scrum Master / QA Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Probar creación de <code>tabla/modelo validations</code> .	P0	Tabla disponible.	Depende de Backend.
2	Probar restricción única de validación duplicada a nivel BD o backend.	P0	Duplicidad prevenida.	Crítico.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
3	Probar GET <code>/incidents/nearby</code> con coordenadas dentro del radio.	P0	Devuelve incidentes cercanos.	Depende de GeoData.
4	Probar GET <code>/incidents/nearby</code> con coordenadas fuera del radio.	P0	No devuelve incidentes lejanos.	Valida distancia.
5	Probar que endpoint nearby no devuelve reportes del mismo usuario.	P0	Autovalidación prevenida desde listado.	Seguridad funcional.
6	Probar que endpoint nearby no devuelve reportes confirmados.	P0	Solo pendientes validables.	Regla central.
7	Probar que la respuesta incluya <code>validation_count</code> .	P0	Conteo disponible.	Frontend lo usa.
8	Registrar bugs de endpoint nearby en tablero.	P0	Bugs visibles.	Corrección miércoles.
9	Actualizar matriz de usuarios demo con resultados reales.	P1	QA trazable.	Preparación del jueves.

Persona 3 — Developer Frontend PWA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Crear pantalla o sección visual “Incidentes cercanos”.	P0	Sección visible.	Base de validación.
2	Capturar ubicación actual para consultar incidentes cercanos.	P0	Lat/Lng disponibles.	Endpoint nearby depende de esto.
3	Crear función <code>fetchNearbyIncidents(lat, lng)</code> .	P0	Función lista para consumir API.	Depende del contrato backend.
4	Mostrar estado “Buscando incidentes cercanos...”.	P1	Feedback visual.	Mejora UX.
5	Mostrar mensaje si no hay incidentes cercanos.	P1	Estado vacío controlado.	UX.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
6	Crear tarjeta visual para cada incidente cercano.	P0	Categoría, estado, distancia y conteo visibles.	Depende de datos API.
7	Integrar componente <code>ValidateButton</code> dentro de cada tarjeta.	P0	Botón visible pero aún no conectado.	Se conectará miércoles.
8	Validar que la sección solo sea visible para usuario autenticado.	P0	Ruta protegida.	Seguridad.
9	Preparar manejo de errores si el usuario niega GPS.	P0	Error claro.	Necesario para nearby.

Persona 4 — Developer Backend/API

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Crear tabla/modelo <code>validations</code> .	P0	Modelo funcional.	Base del sprint.
2	Crear relación <code>Validation.incident_id → Incident.id</code> .	P0	Validación vinculada al incidente.	Necesario para conteo.
3	Crear relación <code>Validation.user_id → User.id</code> .	P0	Validación vinculada al usuario.	Necesario para duplicidad.
4	Implementar restricción única por incidente y usuario.	P0	Impide doble validación.	Regla central.
5	Implementar <code>GET /incidents/nearby</code> .	P0	Endpoint disponible.	Depende de GeoData.
6	Filtrar incidentes por estado "Pendiente".	P0	Solo reportes validables.	Regla funcional.
7	Excluir incidentes creados por el usuario autenticado.	P0	Evita autovalidación en listado.	Seguridad funcional.
8	Incluir <code>validation_count</code> en la respuesta.	P0	Conteo disponible.	Frontend lo necesita.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
9	Documentar endpoint nearby en Swagger/README.	P1	Contrato visible.	Integración.

Persona 5 — Developer GeoData/IA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Implementar función de distancia en metros entre dos coordenadas.	P0	Función operativa.	Base de nearby.
2	Integrar función de distancia con endpoint nearby.	P0	Filtro geográfico activo.	Depende de Backend.
3	Validar incidentes dentro de 300 m con dataset de prueba.	P0	Devuelve casos correctos.	QA lo confirma.
4	Validar incidentes fuera de 300 m con dataset de prueba.	P0	Excluye casos lejanos.	QA lo confirma.
5	Calcular campo opcional <code>distance_meters</code> para cada incidente.	P1	Distancia visible para UI.	Mejora Frontend.
6	Ordenar incidentes cercanos por menor distancia si se aprueba.	P1	Lista más útil.	UX.
7	Revisar rendimiento básico del cálculo con varios incidentes.	P1	Sin lentitud evidente.	RNF rendimiento.
8	Documentar ejemplos de distancia calculada.	P1	Evidencia técnica.	QA y backend.
9	Coordinar con Frontend formato de distancia: metros redondeados.	P1	UI más clara.	Integra visualmente.

Resultado obligatorio del martes

Al cerrar martes debe existir:

- modelo `Validation` creado

- restricción única definida
 - endpoint GET /incidents/nearby funcional
 - función de distancia operativa
 - incidentes cercanos filtrados por radio
 - incidentes propios excluidos del listado
 - frontend con sección de incidentes cercanos preparada
 - pruebas iniciales de nearby ejecutadas
-

Miércoles — Endpoint de validación y conexión del botón

Objetivo del día

Permitir que un ciudadano confirme un incidente cercano y que el sistema registre la validación correctamente.

Persona 1 — Product Owner Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar flujo visual: ver incidente cercano → confirmar → ver actualización.	P0	Flujo aceptado.	Depende de Frontend.
2	Validar mensaje de éxito: “Incidente confirmado correctamente”.	P1	Texto aprobado.	Frontend lo usa.
3	Validar mensaje de duplicidad: “Ya confirmaste este incidente”.	P1	Texto aprobado.	Backend/Frontend lo usan.
4	Validar mensaje de autovalidación si ocurre por intento directo.	P1	Texto aprobado.	Seguridad.
5	Validar que confirmar incidente no cambie aún a “Confirmado” si no llega a 5.	P0	Regla de negocio clara.	Backend la aplica.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
6	Revisar que el conteo suba en uno después de validar.	P0	Criterio aceptado.	QA lo probará.
7	Validar que un incidente confirmado deje de estar en lista cercana pendiente.	P0	Criterio futuro para jueves.	Depende del umbral 5.
8	Revisar avance del sprint y detectar si la regla de 5 está en riesgo.	P0	Riesgos identificados.	Control del sprint.
9	Ajustar prioridad de tareas P1 si el endpoint de validación se retrasa.	P0	Foco en P0.	Protege cierre.

Persona 2 — Scrum Master / QA Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Probar <code>POST /incidents/{id}/validate</code> con usuario válido cercano.	P0	Validación registrada.	Depende de Backend.
2	Probar validación sin token.	P0	Solicitud rechazada.	Seguridad.
3	Probar validación de incidente propio.	P0	Solicitud rechazada.	Autovalidación.
4	Probar validación duplicada del mismo usuario.	P0	Segunda validación rechazada.	Duplicidad.
5	Probar validación de incidente inexistente.	P1	Error controlado.	Calidad API.
6	Probar validación de incidente ya confirmado.	P0	Solicitud rechazada o no permitida.	Regla de estado.
7	Probar que <code>validation_count</code> sube después de validar.	P0	Conteo correcto.	Integra endpoint y listado.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
8	Registrar bugs del endpoint validate.	P0	Bugs visibles.	Corrección jueves.
9	Preparar prueba de cinco usuarios para jueves.	P0	Secuencia de validadores lista.	Cierra Sprint Goal.

Persona 3 — Developer Frontend PWA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Conectar <code>ValidateButton</code> con <code>POST /incidents/{id}/validate</code> .	P0	Botón registra validación.	Depende de Backend.
2	Enviar token del usuario autenticado en la solicitud.	P0	Backend reconoce validador.	Depende Sprint 2.
3	Deshabilitar botón mientras se procesa la validación.	P0	Evita doble click.	Confiabilidad.
4	Mostrar mensaje de éxito al validar.	P1	Feedback claro.	Depende de PO.
5	Mostrar error si el usuario ya validó.	P0	Duplicidad visible.	Depende de backend.
6	Mostrar error si intenta validar algo no permitido.	P0	Error controlado.	UX.
7	Actualizar <code>validation_count</code> en la tarjeta tras validar.	P0	Conteo cambia sin recargar.	UX importante.
8	Refrescar lista nearby después de validar si es necesario.	P1	Datos consistentes.	Mejora.
9	Verificar que la acción funcione en navegador móvil.	P0	PWA funcional.	QA lo valida.

Persona 4 — Developer Backend/API

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Implementar <code>POST /incidents/{id}/validate</code> .	P0	Endpoint funcional.	Tarea central.
2	Proteger endpoint con autenticación.	P0	Solo usuarios logueados validan.	Seguridad.
3	Buscar incidente por ID y validar existencia.	P0	Manejo de incidente inexistente.	Calidad API.
4	Validar que incidente esté en estado "Pendiente".	P0	Solo pendientes se validan.	Regla central.
5	Bloquear si <code>incident.user_id == current_user.id</code> .	P0	Autovalidación impedida.	Regla central.
6	Bloquear si ya existe <code>Validation</code> del mismo usuario para el incidente.	P0	Duplicidad impedida.	Regla central.
7	Crear registro en <code>validations</code> .	P0	Validación guardada.	Resultado principal.
8	Devolver nuevo <code>validation_count</code> .	P0	Frontend actualiza conteo.	Integración.
9	Documentar errores y respuestas del endpoint.	P1	QA y frontend alineados.	Mantenibilidad.

Persona 5 — Developer GeoData/IA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Validar si el usuario que confirma está dentro del radio permitido.	P0	Regla geográfica aplicada o definida.	Puede integrarse al endpoint.
2	Coordinar con Backend si <code>POST validate</code> también validará distancia.	P0	Decisión cerrada.	Evita bypass del frontend.
3	Probar validación con usuario dentro de 300 m.	P0	Validación permitida.	QA lo verifica.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
4	Probar validación con usuario fuera de 300 m.	P0	Validación rechazada si se aplica regla backend.	Seguridad geográfica.
5	Revisar si el frontend manda lat/lng actual al validar.	P0	Datos disponibles para distancia.	Depende Frontend.
6	Ajustar cálculo de distancia si hay discrepancias.	P0	Resultados correctos.	Depende de pruebas.
7	Documentar si la validación geográfica se aplica en nearby, validate o ambos.	P1	Regla clara.	Cierre técnico.
8	Preparar coordenadas para cinco usuarios válidos cercanos.	P0	Prueba de jueves lista.	Cierra umbral 5.
9	Coordinar con QA secuencia exacta de validadores.	P0	Prueba controlada.	Evita errores de demo.

Resultado obligatorio del miércoles

Al cerrar miércoles debe existir:

- endpoint POST /incidents/{id}/validate
- botón de confirmar conectado
- validación registrada en base de datos
- bloqueo de autovalidación
- bloqueo de duplicidad
- conteo de validaciones actualizado
- pruebas con usuario cercano ejecutadas
- prueba de 5 usuarios preparada para jueves

Jueves — Cambio automático a “Confirmado” y actualización visual

Objetivo del día

Completar la regla principal del sprint: cuando cinco usuarios distintos validen un reporte, el incidente cambia automáticamente a **Confirmado** y se refleja en mapa/listado/detalle.

Persona 1 — Product Owner Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar prueba de cinco validaciones distintas.	P0	Criterio de negocio validado.	Cierra Sprint Goal.
2	Confirmar que el estado cambia exactamente al llegar a 5, no antes.	P0	Regla precisa.	Backend lo aplica.
3	Validar que el mapa muestre color rojo al estar Confirmado.	P1	Cambio visual aceptado.	Frontend lo aplica.
4	Validar que el detalle muestre nuevo estado y conteo.	P0	Información actualizada.	Depende de Frontend.
5	Validar que el incidente confirmado ya no aparezca como pendiente cercano.	P0	Regla correcta.	Endpoint nearby.
6	Revisar mensajes finales de confirmación.	P1	Textos aprobados.	UI.
7	Confirmar que no se agrega aún asignación municipal en este sprint.	P0	Alcance protegido.	Sprint 6.
8	Revisar qué historias quedan listas para aceptación.	P0	Cierre preparado.	PO listo para viernes.
9	Definir entrada del Sprint 6: dashboard y duplicados.	P1	Continuidad preliminar.	Próximo sprint.

Persona 2 — Scrum Master / QA Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Ejecutar prueba con usuario A creando reporte pendiente.	P0	Reporte base creado.	Inicio de flujo.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
2	Ejecutar validación con usuario B.	P0	Conteo = 1.	Secuencia.
3	Ejecutar validación con usuario C.	P0	Conteo = 2.	Secuencia.
4	Ejecutar validación con usuario D.	P0	Conteo = 3.	Secuencia.
5	Ejecutar validación con usuario E.	P0	Conteo = 4.	Secuencia.
6	Ejecutar validación con usuario F.	P0	Conteo = 5 y estado Confirmado.	Cierre regla central.
7	Verificar en base de datos que estado sea Confirmado.	P0	Estado confirmado técnicamente.	Evidencia.
8	Verificar que el mapa muestre el nuevo color.	P1	Cambio visual validado.	UX.
9	Verificar que el incidente ya no aparece en nearby como pendiente.	P0	Regla de listado validada.	Crítico.
10	Registrar bugs críticos de cambio de estado.	P0	Correcciones visibles.	Cierre viernes.

Persona 3 — Developer Frontend PWA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Actualizar tarjeta de incidente cercano después de cada validación.	P0	Conteo visible actualizado.	Depende de respuesta backend.
2	Mostrar estado “Confirmado” cuando el backend lo devuelva.	P0	Estado actualizado en UI.	Depende de endpoint.
3	Actualizar color del pin en mapa si el incidente cambia a Confirmado.	P1	Pin rojo.	Conecta con Sprint 4.
4	Actualizar modal de detalle con nuevo conteo y estado.	P0	Detalle consistente.	Importante para demo.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
5	Quitar o deshabilitar botón de confirmar si el incidente ya está Confirmado.	P0	No permite validación posterior.	Regla de estado.
6	Refrescar lista nearby después de que un incidente se confirme.	P0	Sale de pendientes cercanos.	Depende de regla.
7	Corregir errores visuales detectados por QA en validación.	P0	UI estable.	Depende de pruebas.
8	Agregar mensaje visual: “Este incidente ya fue confirmado por la comunidad”.	P1	UX más clara.	Producto.
9	Probar flujo en móvil con usuarios demo.	P0	Validación usable.	QA.
10	Evitar doble click rápido en confirmación.	P0	No duplica solicitud.	Confiabilidad.

Persona 4 — Developer Backend/API

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Implementar conteo posterior a cada validación.	P0	Conteo exacto.	Base del cambio de estado.
2	Implementar regla: si <code>validation_count</code> $\geq$ 5, actualizar estado a Confirmado.	P0	Confirmación automática.	Tarea central.
3	Garantizar que el cambio de estado ocurra dentro del flujo del endpoint validate.	P0	Estado cambia inmediatamente.	Consistencia.
4	Garantizar que usuarios duplicados no cuenten para el umbral.	P0	Solo usuarios distintos.	Regla central.
5	Evitar que validaciones posteriores se creen si el incidente ya está Confirmado.	P0	Estado protegido.	Integridad.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
6	Ajustar GET /incidents/public para devolver conteo actualizado.	P0	Mapa/detalle actualizados.	Frontend depende.
7	Ajustar GET /incidents/nearby para excluir Confirmados.	P0	Lista cercana correcta.	Regla central.
8	Corregir bugs reportados por QA.	P0	Endpoint estable.	Cierre.
9	Actualizar documentación del endpoint validate con regla de 5.	P1	Contrato claro.	Mantenibilidad.
10	Revisar transacción o consistencia para evitar conteos incorrectos.	P1	Menor riesgo de carrera.	Calidad técnica.

Persona 5 — Developer GeoData/IA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Validar ubicación de los cinco usuarios de prueba dentro de 300 m.	P0	Prueba geográfica válida.	Depende de QA.
2	Verificar que validadores fuera del radio no puedan confirmar si la regla se aplica en backend.	P0	Seguridad geográfica.	Importante.
3	Revisar que el endpoint nearby excluya incidentes confirmados aunque estén cerca.	P0	Regla correcta.	Depende Backend.
4	Verificar que la distancia devuelta no afecte el cambio de estado.	P1	Separación de reglas.	Calidad.
5	Ajustar dataset de usuarios si alguna validación falla por distancia.	P0	Prueba de 5 exitosa.	Cierre del sprint.
6	Preparar reporte técnico corto de validación geográfica.	P1	Evidencia técnica.	Sprint Review.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
7	Coordinar con Frontend visualización de distancia si se usa.	P1	UI coherente.	Mejora.
8	Probar escenario: incidente cercano pero propio no validable.	P0	Regla combinada correcta.	Autovalidación.
9	Probar escenario: incidente cercano, pendiente y de otro usuario sí validable.	P0	Flujo válido.	Regla central.
10	Documentar limitación: GPS real puede variar en campo.	P1	Riesgo conocido.	Futuro.

Resultado obligatorio del jueves

Al cerrar jueves debe existir:

- cinco usuarios validando un mismo reporte
- conteo exacto de validaciones
- cambio automático a “Confirmado”
- incidente confirmado excluido de lista nearby
- mapa/detalle mostrando estado actualizado
- bloqueo de validaciones posteriores
- bugs críticos identificados y en corrección

Viernes — Pruebas finales, Sprint Review y cierre

Objetivo del día

Cerrar Sprint 5 demostrando el flujo completo de validación ciudadana cercana.

Persona 1 — Product Owner Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Revisar HU-19: ver incidentes cercanos.	P0	Historia aceptada o rechazada.	Cierre funcional.
2	Revisar HU-20: confirmar incidente.	P0	Historia aceptada o rechazada.	Cierre funcional.
3	Revisar HU-21: bloqueo de autovalidación.	P0	Historia aceptada o rechazada.	Cierre funcional.
4	Revisar HU-22: bloqueo de validación duplicada.	P0	Historia aceptada o rechazada.	Cierre funcional.
5	Revisar HU-23: conteo de validaciones.	P0	Historia aceptada o rechazada.	Cierre funcional.
6	Revisar HU-24: cambio automático a Confirmado.	P0	Historia aceptada o rechazada.	Cierre funcional.
7	Revisar HU-25: actualización visual del mapa si se completó.	P1	Aceptada o trasladada.	Control de alcance.
8	Validar que el incremento realmente reduce reportes falsos o débiles.	P0	Valor del sprint confirmado.	Narrativa del MVP.
9	Definir entrada exacta del Sprint 6: dashboard inicial + duplicados.	P0	Próximo sprint preparado.	Continuidad.
10	Aprobar o rechazar incremento completo del Sprint 5.	P0	Cierre formal.	Scrum Review.

Persona 2 — Scrum Master / QA Técnico

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Ejecutar prueba final: usuario A crea reporte.	P0	Reporte pendiente listo.	Inicio del guion.
2	Ejecutar prueba final: usuarios B-F validan el reporte.	P0	Cinco validaciones registradas.	Cierre central.
3	Verificar que usuario A no pueda validar su reporte.	P0	Autovalidación bloqueada.	Crítico.
4	Verificar que usuario B no pueda validar dos veces.	P0	Duplicidad bloqueada.	Crítico.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
5	Verificar que el estado cambia a Confirmado al llegar a 5.	P0	Regla cumplida.	Sprint Goal.
6	Verificar que el incidente confirmado ya no aparezca como pendiente cercano.	P0	Lista correcta.	Crítico.
7	Verificar que el conteo se vea en detalle.	P0	UI consistente.	HU-23.
8	Verificar que el mapa muestre color actualizado si se implementó.	P1	Visual consistente.	HU-25.
9	Registrar bugs finales y confirmar si son bloqueantes.	P0	Decisión de cierre.	Review.
10	Realizar retrospectiva del Sprint 5.	P1	Mejoras registradas.	Prepara Sprint 6.

Persona 3 — Developer Frontend PWA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Corregir bugs críticos de lista nearby.	P0	Lista estable.	Depende de QA.
2	Corregir bugs críticos del botón validar.	P0	Botón estable.	Depende de QA.
3	Corregir errores de mensajes de éxito/error.	P0	Feedback claro.	UX.
4	Corregir actualización de conteo en UI.	P0	Conteo correcto.	Crítico.
5	Corregir actualización visual de estado Confirmado.	P0	Estado visible.	Cierre.
6	Verificar que usuario no autenticado no pueda validar.	P0	Ruta protegida.	Seguridad.
7	Verificar que el flujo funcione en navegador móvil.	P0	PWA validada.	Demo.

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
8	Preparar demo visual de validación cruzada.	P1	Evidencia para Review.	Presentación.
9	Actualizar README frontend con uso de validaciones.	P1	Documentación mínima.	Cierre técnico.
10	Congelar cambios visuales no críticos.	P1	Evita romper lo estable.	Control de cierre.

Persona 4 — Developer Backend/API

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Corregir bugs críticos de <code>GET /incidents/nearby</code> .	P0	Endpoint estable.	Depende QA.
2	Corregir bugs críticos de <code>POST /incidents/{id}/validate</code> .	P0	Endpoint estable.	Depende QA.
3	Verificar restricción única final en validaciones.	P0	Duplicados imposibles.	Integridad.
4	Verificar bloqueo de autovalidación en backend.	P0	No depende solo del frontend.	Seguridad.
5	Verificar conteo exacto con cinco usuarios.	P0	Conteo confiable.	Cierre central.
6	Verificar cambio automático de estado.	P0	Regla operativa.	Sprint Goal.
7	Verificar que endpoints públicos devuelvan <code>validation_count</code> .	P0	UI consistente.	Frontend.
8	Actualizar documentación API final de validaciones.	P1	Endpoints documentados.	Mantenibilidad.
9	Preparar base de datos para Sprint 6 con incidentes pendientes y confirmados.	P1	Dashboard tendrá datos.	Continuidad.
10	Revisar logs de errores sin datos personales.	P1	Privacidad básica.	RNF seguridad.

Persona 5 — Developer GeoData/IA

N.º	Tarea	Prioridad	Resultado esperado	Relación secuencial
1	Verificar coordenadas finales de usuarios demo.	P0	Pruebas cercanas correctas.	Cierre.
2	Verificar coordenadas finales de incidentes demo.	P0	Cálculo de distancia correcto.	Cierre.
3	Validar escenario dentro de 300 m.	P0	Usuario cercano sí puede validar.	Regla central.
4	Validar escenario fuera de 300 m.	P0	Usuario lejano no ve o no valida.	Regla central.
5	Validar escenario usuario creador dentro de 300 m pero no validable.	P0	Autovalidación bloqueada.	Regla combinada.
6	Confirmar que distancia calculada no tiene errores evidentes.	P0	Resultado confiable.	QA.
7	Documentar dataset final usado en demo.	P1	Evidencia técnica.	Review.
8	Documentar reglas geográficas aplicadas en validación.	P1	Trazabilidad.	Cierre.
9	Preparar casos de duplicados cercanos para Sprint 6.	P1	Entrada del siguiente sprint.	Continuidad.
10	Coordinar con Backend datos necesarios para dashboard geográfico.	P1	Sprint 6 preparado.	Próximo incremento.

Resultado obligatorio del viernes

Al cerrar viernes, el equipo debe poder demostrar:

1. Usuario A crea un reporte pendiente.
2. Usuario A no puede validar su propio reporte.
3. Usuario B ve el reporte cercano y lo valida.

4. Usuario B no puede validarlo dos veces.
 5. Usuarios C, D, E y F también validan el reporte.
 6. El conteo llega a cinco validaciones.
 7. El sistema cambia el estado a **Confirmado** automáticamente.
 8. El incidente confirmado deja de aparecer como pendiente validable.
 9. El detalle muestra el nuevo estado y el conteo.
 10. El mapa refleja el estado actualizado si la actualización visual fue completada.
-

Definition of Done del Sprint 5

Criterio	Obligatorio
Modelo Validation creado	Sí
Validación vinculada a usuario e incidente	Sí
GET /incidents/nearby funciona	Sí
Nearby filtra por radio de 300 m	Sí
Nearby excluye reportes propios	Sí
Nearby muestra solo reportes pendientes	Sí
POST /incidents/{id}/validate funciona	Sí
Endpoint requiere usuario autenticado	Sí
Backend bloquea autovalidación	Sí
Backend bloquea validación duplicada	Sí
Conteo de validaciones funciona	Sí
Estado cambia a Confirmado con 5 validaciones	Sí
Frontend muestra botón de confirmar	Sí
Frontend muestra conteo actualizado	Sí
QA probó cinco usuarios distintos	Sí
Bugs críticos corregidos	Sí
Sprint Review realizado	Sí

Incremento esperado al final del Sprint 5

QHALI MVP – Incremento Sprint 5

- └─ Modelo Validation
- └─ GET /incidents/nearby
- └─ POST /incidents/{id}/validate
- └─ Filtro por radio de 300 m
- └─ Exclusión de reportes propios
- └─ Bloqueo de autovalidación
- └─ Bloqueo de validación duplicada
- └─ Conteo de validaciones
- └─ Cambio automático a Confirmado con 5 validaciones
- └─ Vista de incidentes cercanos
- └─ Botón Confirmar incidente
- └─ Actualización visual del conteo
- └─ Pruebas con múltiples usuarios

Sprint Review del Sprint 5

La demo debe seguir este guion:

1. Iniciar sesión como Usuario A.
2. Crear reporte con foto, categoría, descripción y GPS.
3. Confirmar que el reporte aparece como Pendiente.
4. Iniciar sesión como Usuario B cerca del incidente.
5. Ver el reporte en “Incidentes cercanos”.
6. Presionar “Confirmar incidente”.
7. Ver conteo = 1.
8. Intentar validar otra vez con Usuario B.
9. Confirmar que el sistema lo bloquea.
10. Iniciar sesión con Usuarios C, D, E y F.
11. Validar el mismo incidente.
12. Confirmar que al llegar a 5 validaciones el estado cambia a Confirmado.
13. Ver que el incidente ya no aparece como pendiente validable.

Retrospectiva del Sprint 5

Preguntas mínimas:

1. ¿Qué fue más difícil: distancia, validaciones o cambio automático de estado?
2. ¿La regla de cinco validaciones se implementó de forma segura?

3. ¿El sistema bloquea correctamente autovalidación y duplicidad?
 4. ¿El flujo de validación es comprensible para un ciudadano común?
 5. ¿Qué datos quedan listos para construir el dashboard del Sprint 6?
-

Entrada directa para Sprint 6

Para empezar el Sprint 6 deben quedar listos:

1. Reportes pendientes y confirmados.
2. Validaciones registradas.
3. Conteo de validaciones por incidente.
4. Estados actualizados.
5. Datos suficientes para indicadores.
6. Incidentes con coordenadas válidas.
7. Usuarios demo con reportes y validaciones.

El **Sprint 6** debe enfocarse en:

Dashboard inicial de gestión
Filtros por estado y categoría
Cambio manual de estado
Indicadores básicos
Detección básica de posibles duplicados